

Институт перспективных технологий и индустриального
программирования
Кафедра «Индустриальное программирование»

SRS – Software Requirements Specification для проекта «Визуализатор алгоритмов сортировки»

Берг Ксения Вячеславовна

ЭФБО-02-24

Оглавление

Глава 1. Введение	3
1.1 Цель.....	3
1.2 Область действия.....	3
1.3. Определения и аббревиатуры.....	4
Глава 2. Общее описание.	5
2.1. Описание продукта.....	5
2.2. Функции продукта	5
2.3. Характеристики пользователей.....	6
Глава 3. Детальные требования	7
3.1. Методология сбора требований	7
3.2. Бизнес-требования.....	8
3.3. Пользовательские требования.....	8
3.4. Функциональные требования.....	9
3.5. Нефункциональные требования.....	10
Глава 4. UML диаграммы.....	12
4.1. Диаграмма вариантов использования (Use Case Diagram).....	12
4.2. Диаграмма классов (Class Diagram)	13
4.3. Диаграмма состояний (Sequence diagram)	14

Глава 1. Введение

1.1 Цель

Настоящий документ представляет собой спецификацию требований к программному обеспечению (Software Requirements Specification, SRS) для приложения «Визуализатор алгоритмов сортировки». Документ предназначен для согласования ожиданий между заказчиком, аналитиками, разработчиками и тестировщиками, а также служит основой для последующего проектирования, реализации и проверки качества программного продукта.

Целью разработки является создание веб-ориентированного приложения для наглядной демонстрации работы алгоритмов сортировки. Приложение ориентировано на использование в образовательных целях - для изучения принципов работы алгоритмов студентами. Продукт должен обеспечивать возможность наблюдения за процессом сортировки в реальном времени, управления его параметрами и анализа эффективности различных методов.

1.2 Область действия

В область действия разрабатываемой системы входят следующие компоненты и функции:

- Визуализация пяти алгоритмов сортировки: пузырьковая (Bubble Sort), шейкерная (Shaker Sort), быстрая (Quick Sort), слиянием (Merge Sort), вставками (Insertion Sort);
- Генерация числового массива с настраиваемыми параметрами (размер от 5 до 50 элементов, значения от 0 до 100, тип начального распределения: случайный, отсортированный, обратный, почти отсортированный);
- Управление процессом сортировки (запуск, пауза, остановка, пошаговое выполнение, регулировка скорости анимации);
- Отображение дополнительной информации: временная сложность алгоритма, количество сравнений и перестановок, подсветка текущих строк псевдокода, справка о практическом применении;
- Звуковое сопровождение процесса сортировки (тональные сигналы при сравнении и перестановке элементов) с возможностью отключения;
- Пользовательский интерфейс, обеспечивающий интуитивно понятное взаимодействие и доступность на различных устройствах через браузер.

Следующие функции и возможности остаются за пределами текущей версии продукта:

- Реализация алгоритмов сортировки для нечисловых данных;
- Сохранение истории сессий пользователя;

- Нативное мобильное приложение (доступ только через веб-браузер).

1.3. Определения и аббревиатуры

В ниже представленной таблице 1 указаны все специфичные для документа аббревиатуры.

Таблица 1 – Определения и аббревиатуры

Термин	Определение
Алгоритм сортировки	Последовательность операций, упорядочивающая элементы массива по возрастанию (в контексте данного продукта)
Визуализация	Графическое представление процесса сортировки, где каждый элемент массива отображается в виде столбца, высота которого пропорциональна значению элемента
Сравнение	Операция, в ходе которой алгоритм определяет отношение между двумя элементами (больше, меньше или равно)
Перестановка (своп)	Операция обмена двух элементов массива местами
Пошаговый режим	Режим выполнения, при котором каждый следующий шаг алгоритма (сравнение или перестановка) инициируется пользователем нажатием кнопки
Псевдокод	Упрощённое описание алгоритма на естественном языке с элементами языка программирования, не привязанное к конкретной реализации
Временная сложность	Характеристика зависимости времени выполнения алгоритма от размера входных данных, обозначаемая как $O(n)$ (О-большое)
Сонарификаци	Преобразование данных в звук; в контексте приложения — звуковое сопровождение операций сравнения и перестановки
SRS	Software Requirements Specification — документ, фиксирующий все требования к программному продукту

Глава 2. Общее описание.

2.1. Описание продукта

Программный продукт «Визуализатор алгоритмов сортировки» представляет собой интерактивный инструмент, предназначенный для наглядной демонстрации работы различных алгоритмов упорядочивания данных. Приложение ориентировано на использование в образовательных целях — в частности, для изучения принципов работы алгоритмов студентами, а также для проведения лекционных демонстраций преподавателями.

Ключевая ценность продукта заключается в возможности увидеть, «как думает» алгоритм: пользователь наблюдает за процессом сортировки в реальном времени, отслеживая каждое сравнение и каждую перестановку элементов. В отличие от статичных схем и текстовых описаний, которые не передают динамику процесса, визуализация позволяет понять внутреннюю логику алгоритма, увидеть его сильные и слабые стороны на различных типах входных данных.

Продукт не требует установки и доступен на любом устройстве с современным веб-браузером, что делает его удобным инструментом как для самостоятельного изучения материала вне аудитории, так и для использования в учебных аудиториях.

2.2. Функции продукта

Разрабатываемое приложение предоставляет пользователю следующие группы функций:

1. Управление массивом данных

- Генерация числового массива с настраиваемыми параметрами (размер от 5 до 50 элементов, значения в диапазоне от 0 до 100);
- Выбор типа начального распределения элементов: случайный, отсортированный, обратный, почти отсортированный;
- Визуализация массива в виде столбчатой диаграммы, где высота каждого столбца пропорциональна значению элемента.

2. Управление сортировкой

- Выбор одного из пяти алгоритмов: пузырьковая, шейкерная, быстрая, слиянием, вставками;
- Запуск, пауза и остановка выполнения алгоритма;
- Пошаговый режим, позволяющий вручную контролировать каждый шаг сортировки;
- Регулировка скорости анимации с помощью ползунка.

3. Информационная поддержка

- Отображение временной сложности выбранного алгоритма (О-нотация);
- Подсчёт и отображение количества выполненных сравнений и перестановок в реальном времени;
- Подсветка текущих строк псевдокода, синхронизированная с выполняемыми действиями;
- Справка о практическом применении каждого алгоритма (в каких библиотеках и сферах используется).

4. Дополнительные возможности

- Звуковое сопровождение операций сравнения и перестановки с возможностью отключения;
- Приветственная страница с общей информацией о принципах работы алгоритмов сортировки и рекомендациями по использованию приложения.

2.3. Характеристики пользователей

Целевая аудитория продукта может быть разделена на две основные группы (Таблица 2):

Таблица 2 – Характеристики целевой аудитории

Группа	Описание	Потребности
Обучающиеся	Студенты, школьники старших классов, а также все, кто самостоятельно изучает алгоритмы и структуры данных. Имеют базовое понимание принципов программирования или находятся в процессе его формирования	Наглядное объяснение, возможность замедлить и детально разобрать каждый шаг, связь визуализации с теоретическим описанием, наглядное сравнение эффективности различных алгоритмов
Преподаватели	Используют приложение для демонстрации на лекциях, практических занятиях или для подготовки учебных материалов	Быстрое переключение между алгоритмами, возможность акцентировать внимание на ключевых моментах, сравнительный анализ поведения алгоритмов на разных входных данных, понятный интерфейс для демонстрации аудитории

Технические навыки: пользователь должен обладать базовым пониманием алгоритмов и принципов программирования для осмысленного восприятия псевдокода и анализа процесса сортировки.

Глава 3. Детальные требования

3.1. Методология сбора требований

Сбор требований к разрабатываемой системе осуществлялся с использованием комбинации методов, позволяющих наиболее полно выявить потребности целевой аудитории и определить функциональные границы продукта.

Анализ аналогов

Были изучены существующие решения в области визуализации алгоритмов: Visualgo.net, sortvisualizer.com. Анализ позволил:

- выявить функциональные возможности, которые ожидают пользователи (выбор алгоритмов, регулировка скорости анимации, подсветка сравниваемых элементов);
- определить недостатки существующих аналогов (отсутствие синхронизации с псевдокодом, ограниченная настройка входных данных, невозможность пошагового выполнения у некоторых решений);
- сформировать понимание оптимального пользовательского интерфейса и набора элементов управления.

User Stories

Для фокусировки на пользовательской ценности каждой функции требования были сформулированы в виде пользовательских историй. Такой подход позволил:

- четко определить, какие задачи решает каждый элемент функциональности;
- обеспечить приоритизацию требований с точки зрения реальных потребностей обучающихся и преподавателей;
- избежать включения избыточных функций, не приносящих прямой пользы целевой аудитории.

Мозговой штурм

В процессе проработки концепции продукта были сгенерированы дополнительные идеи, расширяющие функциональность и повышающие образовательную ценность приложения:

- звуковое сопровождение процесса сортировки;
- приветственная страница с общей информацией о принципах работы алгоритмов;
- подсветка строк псевдокода, синхронизированная с анимацией;
- справка о практическом применении каждого алгоритма.

3.2. Бизнес-требования

Бизнес-требования определяют высокоуровневые цели разработки и отвечают на вопрос «Зачем создаётся данный продукт?». Они отражают ценность системы для заказчика и образовательного процесса в целом.

Таблица 3 – Бизнес-требования

ID	Требование
БР-1	Обеспечить наглядную демонстрацию работы алгоритмов сортировки для использования в образовательных целях
БР-2	Предоставить пользователю возможность интерактивного взаимодействия с процессом сортировки
БР-3	Обеспечить возможность гибкой конфигурации входных данных (размер массива, тип начального распределения элементов) для анализа поведения алгоритмов в различных сценариях — от лучшего до худшего случая
БР-4	Упростить восприятие сложных алгоритмов за счёт визуализации, анимации и синхронизации с псевдокодом

3.3. Пользовательские требования

Пользовательские требования описывают задачи, которые пользователи решают с помощью системы. Они сформулированы в виде пользовательских историй (User Stories) по шаблону «Как <роль>, я хочу <действие>, чтобы <ценность>», что позволяет сохранить фокус на потребностях целевой аудитории.

Таблица 4 – Пользовательские требования

ID	Требование
ПТ-1	Пользователь может выбрать один из пяти алгоритмов сортировки для запуска
ПТ-2	Пользователь может сгенерировать новый массив с заданными параметрами (размер, тип распределения)
ПТ-3	Пользователь может наблюдать анимацию перестановок элементов в реальном времени
ПТ-4	Пользователь может управлять скоростью анимации с помощью ползунка
ПТ-5	Пользователь может приостановить выполнение, остановить сортировку или выполнить алгоритм пошагово
ПТ-6	Пользователь видит текущие сравниваемые и перемещаемые элементы за счёт цветовой подсветки
ПТ-7	Пользователь видит статистику выполнения (количество сравнений, перестановок) и временную сложность алгоритма
ПТ-8	Пользователь видит псевдокод алгоритма с подсветкой текущей выполняемой строки

ПТ-9	Пользователь может ознакомиться со справкой о практическом применении каждого алгоритма
ПТ-10	Пользователь может включить или отключить звуковое сопровождение процесса сортировки

3.4. Функциональные требования

Функциональные требования определяют конкретные действия, которые система должна выполнять. Они детализируют пользовательские требования и служат основой для разработки и тестирования. Для удобства навигации требования сгруппированы по функциональным модулям.

3.4.1. Управление массивом

Таблица 5 – Функциональные требования: управление массивом

ID	Требование
ФТ-1.1	Генерация массива целых чисел размером от 5 до 50 элементов, значения в диапазоне от 0 до 100
ФТ-1.2	Выбор типа начального распределения элементов: случайный, отсортированный, отсортированный в обратном порядке, почти отсортированный
ФТ-1.3	Визуализация массива в виде столбчатой диаграммы, где высота столбца пропорциональна значению элемента

3.4.2. Управление сортировкой

Таблица 6 – Функциональные требования: управление сортировкой

ID	Требование
ФТ-2.1	Выбор алгоритма из пяти доступных: пузырьковая, шейкерная, быстрая, слиянием, вставками
ФТ-2.2	Запуск выбранного алгоритма с текущим массивом
ФТ-2.3	Пауза выполнения с возможностью последующего возобновления
ФТ-2.4	Остановка выполнения с возвратом массива в исходное состояние
ФТ-2.5	Пошаговый режим: выполнение одного шага (сравнения или перестановки) по нажатию кнопки
ФТ-2.6	Регулировка скорости анимации с помощью ползунка. Диапазон регулировки должен охватывать: минимальную скорость, обеспечивающую комфортное наблюдение за каждым шагом (не менее 0.5 секунды на шаг), и максимальную скорость, позволяющую оценить общую динамику (не более 0.02 секунды на шаг)

3.4.3. Визуализация и информационная поддержка

Таблица 7 – Функциональные требования: визуализация и информационная поддержка

ID	Требования
----	------------

ФТ-3.1	Сравниваемые в текущем шаге элементы должны визуально выделяться на фоне остальных (например, изменением цвета, обводкой или анимацией). Способ выделения должен быть единообразным для всех алгоритмов
ФТ-3.2	Перемещаемые (меняющиеся местами) элементы должны визуально выделяться иным способом, отличным от выделения сравниваемых элементов, чтобы пользователь мог однозначно различать эти типы операций
ФТ-3.3	Отображение текущего количества сравнений и перестановок в реальном времени
ФТ-3.4	Отображение временной сложности выбранного алгоритма в формате О-нотации
ФТ-3.5	Отображение псевдокода алгоритма с подсветкой текущей выполняемой строки, синхронизированной с анимацией
ФТ-3.6	Отображение справки о практическом применении алгоритма при его выборе
ФТ-3.7	Звуковое сопровождение: различные тональные сигналы для сравнения и перестановки элементов
ФТ-3.8	Кнопка (переключатель) для отключения звукового сопровождения

3.4.4. Пользовательский интерфейс

Таблица 8 – Функциональные требования: пользовательский интерфейс

ID	Требования
ФТ-4.1	Приветственная страница с кратким описанием принципов работы алгоритмов сортировки и рекомендациями по использованию приложения
ФТ-4.2	Русскоязычный интерфейс с подписями всех элементов управления
ФТ-4.3	Подсказки при наведении курсора на элементы управления

3.5. Нефункциональные требования

Нефункциональные требования определяют критерии качества системы — как хорошо, быстро и надёжно она должна работать. Они задают ограничения и характеристики, которым должно соответствовать решение.

Таблица 9 – Нефункциональные требования

ID	Категория	Требование
НФТ-1	Производительность	Анимация должна поддерживать плавное отображение (не менее 24 кадров в секунду) при размере массива до 50 элементов на типичном компьютере или ноутбуке
НФТ-2	Удобство использования	Интерфейс должен быть интуитивно понятным: пользователь должен понимать

		назначение каждого элемента управления без чтения инструкции
НФТ-3	Надёжность	Приложение не должно завершаться аварийно или зависать при некорректных действиях пользователя (например, при попытке запуска сортировки на пустом массиве)
НФТ-4	Расширяемость	Архитектура приложения должна позволять добавлять новые алгоритмы сортировки без изменения существующего кода визуализации и интерфейса
НФТ-5	Доступность	Приложение должно быть доступно на любом устройстве с современным веб-браузером (последние версии Chrome, Firefox, Safari, Edge) без необходимости установки дополнительного программного обеспечения

Глава 4. UML диаграммы

Для визуального моделирования разрабатываемой системы были использованы диаграммы Unified Modeling Language (UML), позволяющие представить архитектуру программного продукта с различных точек зрения. В рамках спецификации требований представлены три типа диаграмм: диаграмма вариантов использования, диаграмма классов и диаграмма последовательности.

4.1. Диаграмма вариантов использования (Use Case Diagram)

Актор системы: Пользователь — обобщённая роль, объединяющая всех, кто взаимодействует с приложением (обучающиеся, преподаватели, самостоятельно изучающие алгоритмы).

Основные прецеденты: управление массивом, выбор алгоритма, управление сортировкой, просмотр визуализации, просмотр статистики, просмотр псевдокода, просмотр справки, управление звуком.

Отношения между прецедентами:

- «Управление сортировкой» включает («include») «Просмотр визуализации», «Просмотр статистики» и «Просмотр псевдокода» — запуск сортировки неразрывно связан с визуальным отображением, статистикой и синхронизированным псевдокодом.
- «Выбор алгоритма» расширяет («extend») «Просмотр справки» — справка доступна после выбора алгоритма, но не является обязательным действием.

Диаграмма вариантов использования показывает, что управление сортировкой обязательно включает визуализацию, статистику и псевдокод, а просмотр справки является опциональным расширением выбора алгоритма.

Диаграмма вариантов использования представлена на рисунке 1.

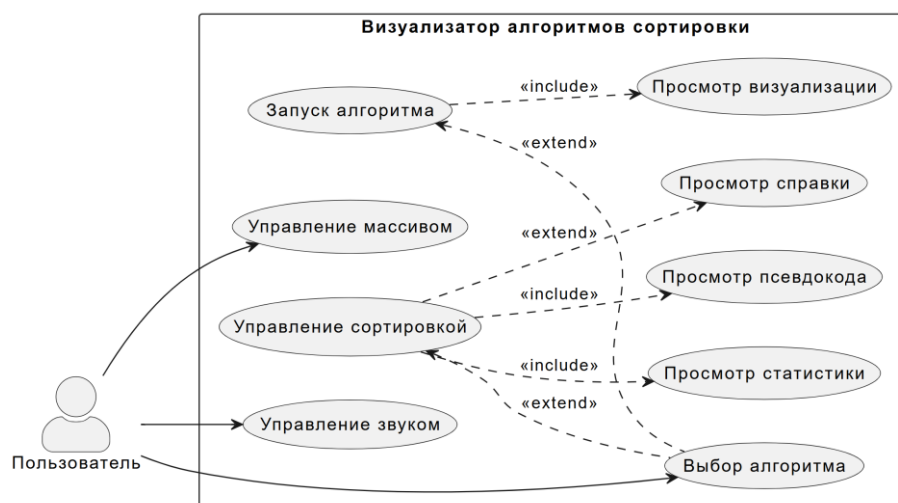


Рисунок 1 – Use Case диаграмма

4.2. Диаграмма классов (Class Diagram)

Диаграмма классов представляет статическую структуру системы. Разработанная архитектура обеспечивает расширяемость: добавление нового алгоритма требует только создания класса-наследника без изменения существующего кода.

Основные классы:

- `SortingAlgorithm` (абстрактный) — базовый класс для всех алгоритмов. Содержит защищённые атрибуты `comparisons`, `swaps`, `array` и абстрактный метод `sort()`.
- `BubbleSort`, `ShakerSort`, `QuickSort`, `MergeSort`, `InsertionSort` — конкретные реализации алгоритмов, наследующие `SortingAlgorithm`.
- `ArrayVisualizer` — центральный контроллер, управляющий массивом, анимацией и пользовательским интерфейсом.
- `Statistics` — учёт и отображение количества сравнений и перестановок.
- `PseudoCodeDisplay` — отображение псевдокода с подсветкой текущей строки.
- `SoundManager` — управление звуковым сопровождением.
- `DistributionType` (enum) — перечисление типов начального распределения массива: случайный, отсортированный, обратный, почти отсортированный.

Отношения между классами:

- Наследование: все конкретные алгоритмы наследуют абстрактный класс `SortingAlgorithm`.
- Композиция: `ArrayVisualizer` содержит `Statistics`, `PseudoCodeDisplay` и `SoundManager`.
- Ассоциация: `ArrayVisualizer` использует текущий выбранный алгоритм.

Диаграмма классов представлена на рисунке 2

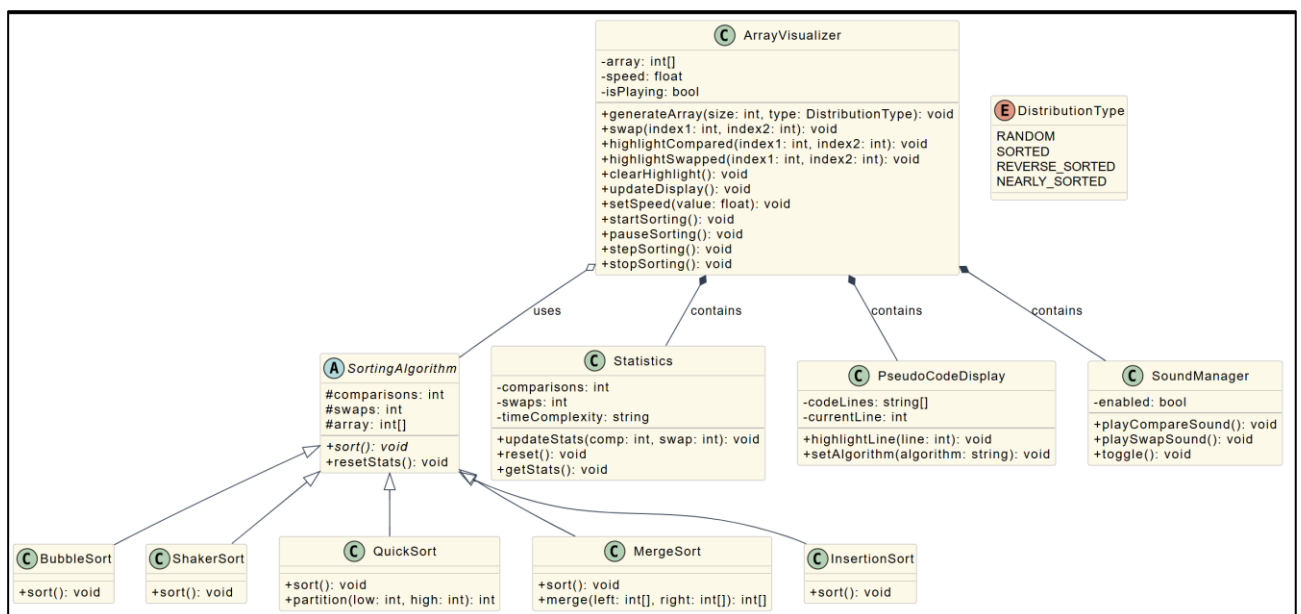


Рисунок 2 – Class diagram

4.3. Диаграмма состояний (Sequence diagram)

Диаграмма последовательности детализирует сценарий «Запуск и выполнение сортировки с пошаговым контролем», показывая взаимодействие компонентов во времени.

Участники: Пользователь, UI, ArrayVisualizer, SortingAlgorithm, Statistics, PseudoCodeDisplay, SoundManager.

Ключевые этапы сценария:

1. Пользователь выбирает алгоритм и генерирует массив.
2. При запуске сортировки ArrayVisualizer сбрасывает статистику и запускает sort() выбранного алгоритма.
3. На каждом шаге алгоритм уведомляет ArrayVisualizer о сравнениях и перестановках, который обновляет интерфейс, статистику, псевдокод и воспроизводит звуковые сигналы.
4. Пользователь может приостановить выполнение, выполнить шаг или остановить сортировку с возвратом массива в исходное состояние.
5. По завершении выводится итоговая статистика.

Диаграмма последовательности представлена на рисунке 3.

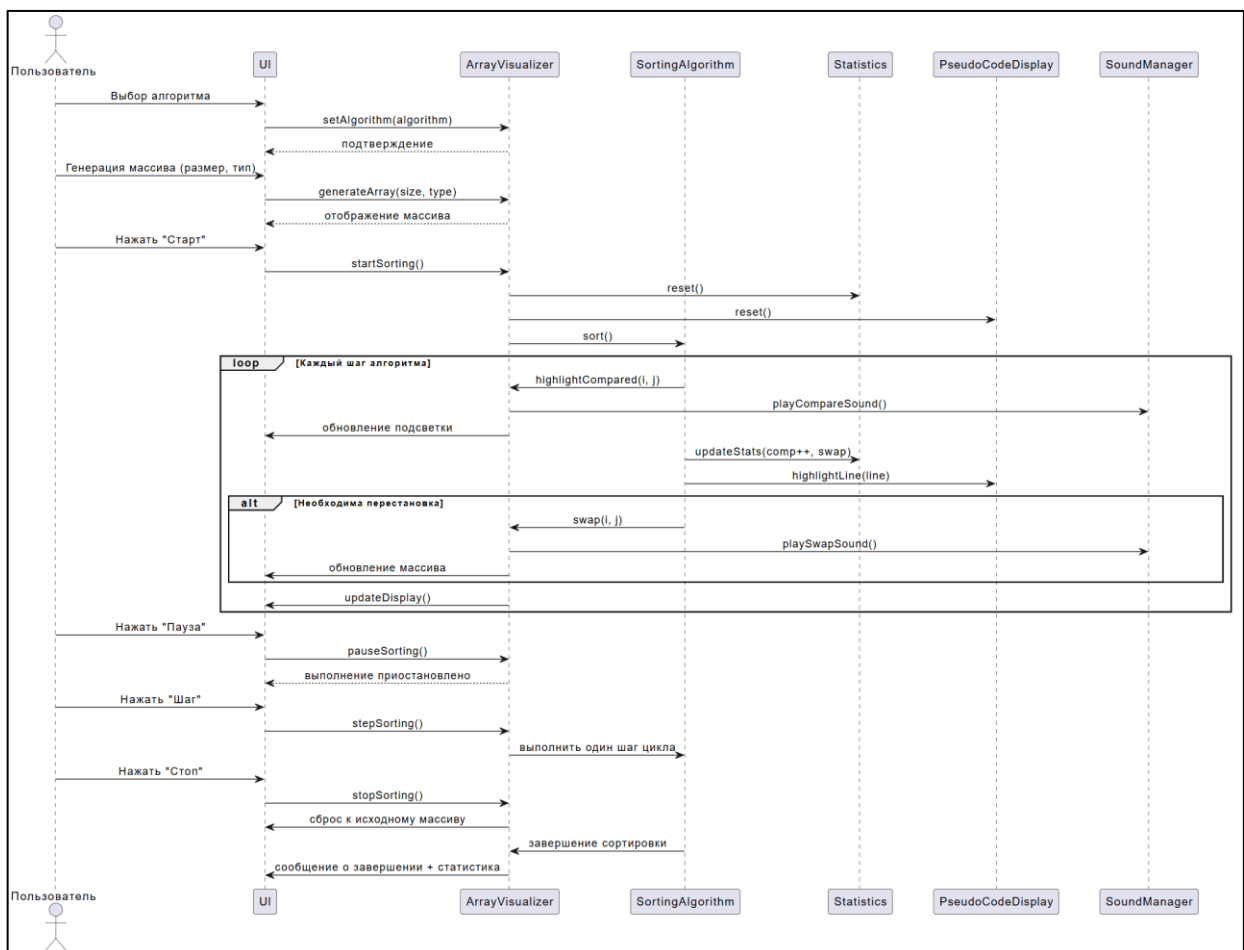


Рисунок 3 – Sequence diagram

Глава 5. Паттерны проектирования

В данной главе представлен анализ применимости паттернов проектирования к архитектуре разрабатываемого приложения «Визуализатор алгоритмов сортировки». Для каждой ключевой архитектурной задачи рассмотрены как выбранный паттерн, так и его ближайшая альтернатива, с обоснованием принятого решения.

5.1. Методология выбора паттернов

Выбор паттернов производился на основе следующих критериев, вытекающих из требований, сформулированных в Главе 3:

- **Расширяемость (НФТ-4):** Архитектура должна позволять добавление новых алгоритмов сортировки без модификации существующего кода визуализации и интерфейса.
- **Слабая связность:** Модули алгоритмов, визуализации, статистики и пользовательского интерфейса должны взаимодействовать через чётко определённые интерфейсы, а не через прямые ссылки друг на друга.
- **Нативное использование Qt:** Поскольку в качестве фреймворка выбран Qt, приоритет отдаётся паттернам, имеющим встроенную поддержку в данном фреймворке (в частности, Signals & Slots как реализация Наблюдателя).

5.2. Паттерн «Стратегия»

Решаемая задача

Приложение должно предоставлять пользователю возможность выбора одного из пяти алгоритмов сортировки (ФТ-2.1). При этом модуль визуализации (ArrayVisualizer) не должен содержать условной логики, зависящей от конкретного алгоритма.

Суть паттерна

Паттерн «Стратегия» определяет семейство взаимозаменяемых алгоритмов, инкапсулирует каждый из них в отдельный класс и делает их взаимозаменяемыми. Модуль визуализации работает с интерфейсом стратегии, не завися от конкретной реализации.

Реализация в проекте

1. Создаётся абстрактный базовый класс ISortingAlgorithm, объявляющий чисто виртуальный метод sort(QVector<int>& data).
2. Конкретные классы BubbleSort, ShakerSort, QuickSort, MergeSort, InsertionSort наследуют ISortingAlgorithm и предоставляют собственные реализации метода sort().

3. Класс `ArrayVisualizer` хранит указатель типа `ISortingAlgorithm*` на текущий выбранный алгоритм. При выборе пользователем другого алгоритма указатель заменяется на новый объект-стратегию.

Альтернативный паттерн

Ближайшей альтернативой «Стратегии» является «Шаблонный метод». Однако в данном проекте он неприменим, потому что предполагает наличие общего скелета алгоритма в базовом классе, при этом подклассы переопределяют лишь отдельные шаги. Данный подход неприменим к рассматриваемому набору алгоритмов, поскольку они фундаментально различаются по структуре: `QuickSort` и `MergeSort` используют рекурсивное разбиение, `BubbleSort` и `ShakerSort` - итеративный обход с вложенными циклами, `InsertionSort` - построение отсортированной подпоследовательности. У этих алгоритмов отсутствует общий каркас, который можно было бы вынести в базовый класс.

5.3. Паттерн «Фабричный метод»

Решаемая задача

Создание конкретного экземпляра класса-алгоритма на основе выбора пользователя в выпадающем списке. Логика создания должна быть изолирована от пользовательского интерфейса, чтобы при добавлении новых алгоритмов не требовалось модифицировать модуль визуализации и управления сортировкой (`ArrayVisualizer`).

Суть паттерна

Паттерн «Фабричный метод» позволяет делегировать логику создания объекта отдельной функции или классу, изолируя клиентский код от знания конкретных типов.

Реализация в проекте:

- Создаётся класс `AlgorithmFactory` со статическим методом: `static ISortingAlgorithm* createAlgorithm(const QString& id);`
- Внутри метода осуществляется сопоставление строкового идентификатора с конкретным классом и возвращается указатель на соответствующий объект-стратегию.
- Код главного окна вызывает данный метод, передавая значение, полученное из `QComboBox`.

Альтернативный подход

Классическая форма «Фабричного метода» предполагает создание иерархии фабрик (абстрактный `AlgorithmFactory` и подклассы `BubbleSortFactory`, `QuickSortFactory` и т.д.), где каждый подкласс отвечает за создание одного типа алгоритма. В контексте данного проекта логика создания всех пяти алгоритмов идентична (вызов конструктора без параметров), поэтому иерархия из шести классов фабрик была бы избыточной. Параметризованная форма

обеспечивает необходимую изоляцию клиентского кода (ArrayVisualizer) от конкретных типов алгоритмов при минимальной сложности реализации. Принцип открытости/закрытости для новых алгоритмов обеспечивается на уровне паттерна «Стратегия» — ArrayVisualizer не требует изменений при добавлении нового класса-алгоритма.

5.4. Паттерн «Наблюдатель»

Решаемая задача

В процессе выполнения алгоритм сортировки должен оповещать пользовательский интерфейс о происходящих событиях — сравнениях и перестановках элементов. При этом классы алгоритмов не должны содержать прямых вызовов методов визуализации, чтобы сохранялась возможность замены интерфейсной части без модификации логики сортировки.

Суть паттерна

Паттерн «Наблюдатель» определяет отношение «один ко многим» между издателем и подписчиками. При изменении состояния издателя все подписчики автоматически оповещаются.

Реализация в проекте

Фреймворк Qt предоставляет встроенную реализацию данного паттерна через механизм Signals & Slots. Класс ISortingAlgorithm наследует QObject и объявляет сигналы:

- compared(int index1, int index2) — испускается при сравнении двух элементов;
- swapped(int index1, int index2) — испускается при перестановке двух элементов местами.

Подписчиками являются:

- ArrayVisualizer — подписывается на сигналы и обновляет цветовую подсветку столбцов диаграммы;
- Statistics — подписывается для инкремента счётчиков сравнений и перестановок;
- SoundManager — подписывается для воспроизведения тональных сигналов;
- PseudoCodeDisplay — подписывается для подсветки текущей строки псевдокода.

Альтернативный паттерн

Паттерн «Посредник» оправдан, когда множество компонентов взаимодействуют по сложной схеме «каждый с каждым», и централизация этой логики в одном классе-диспетчере упрощает систему. В архитектуре данного приложения взаимодействие имеет простую однонаправленную топологию: один издатель (алгоритм сортировки) рассылает уведомления нескольким подписчикам (визуализация, статистика, звук, псевдокод). Обратные вызовы от подписчиков к алгоритму отсутствуют, и подписчики не взаимодействуют друг с другом. Введение Посредника в такой топологии привело бы к появлению дополнительного класса-

прослойки без снижения связности системы. Кроме того, механизм Signals & Slots фреймворка Qt является прямой и эффективной реализацией паттерна Observer и не требует написания инфраструктурного кода для управления подписками.

5.5. Паттерн «Команда»

Решаемая задача

Приложение должно поддерживать пошаговый режим выполнения сортировки. Для повышения образовательной ценности целесообразно предусмотреть возможность не только пошагового продвижения вперёд, но и отката на один или несколько шагов назад, чтобы обучающийся мог повторно просмотреть ключевые моменты алгоритма.

Суть паттерна

Паттерн «Команда» инкапсулирует запрос на выполнение действия в отдельный объект, позволяя сохранять историю операций, отменять их и выстраивать очереди выполнения.

Реализация в проекте

- Создаётся абстрактный базовый класс `Command` с чисто виртуальными методами `execute()` и `undo()`.
- Конкретные подклассы `CompareCommand` и `SwapCommand` хранят ссылку на массив и индексы затронутых элементов. Метод `execute()` выполняет операцию (сравнение или перестановку) и испускает соответствующий сигнал для обновления UI. Метод `undo()` возвращает массив в состояние до операции.
- Класс `SortingController` хранит стек `QStack<Command*>` выполненных команд. При нажатии кнопки «Шаг вперёд» очередная команда исполняется и помещается в стек. При нажатии «Шаг назад» последняя команда извлекается и отменяется.

Альтернативный паттерн

Паттерн «Хранитель» сохраняет полный снимок состояния объекта. Для задачи пошагового отката это было бы расточительным решением: при каждом шаге пришлось бы копировать весь массив целиком (до 50 элементов), тогда как команда хранит лишь индексы изменённых элементов. Кроме того, Хранитель не несёт информации о том, какое именно действие было выполнено - он лишь фиксирует «было» и «стало», что обедняет информационную поддержку (нельзя подсветить, какая строка псевдокода соответствует откатываемому шагу). Паттерн «Команда» сохраняет семантику операции, что лучше согласуется с образовательными целями приложения.